

MODULE 20

Big Data and Cloud Computing

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Big Data and Cloud Computing

What is Big Data?

Big Data: datasets too large for traditional tools.

5 V's of Big Data:

Volume: terabytes to petabytes

Velocity: speed of data generation

Variety: structured, unstructured, semi-structured

Veracity: data quality and accuracy

Value: business insights extracted

Tools: Hadoop, Spark, Kafka, Hive, HBase

Apache Spark with PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

# Create Spark session
spark = SparkSession.builder.appName('DataScience').getOrCreate()

# Load data
df = spark.read.csv('large_data.csv', header=True, inferSchema=True)

# Transformations (lazy evaluation)
df_filtered = df.filter(col('age') > 25)
df_grouped = df_filtered.groupBy('department').agg(
    avg('salary').alias('avg_salary'),
    count('*').alias('count')
)

# Action (triggers execution)
df_grouped.show()

# ML with Spark MLlib
from pyspark.ml.classification import RandomForestClassifier
```

Cloud Platforms

AWS (Amazon Web Services):

S3: object storage

EC2: virtual machines

SageMaker: ML platform

Redshift: data warehouse

Google Cloud Platform:

BigQuery: serverless analytics

Vertex AI: ML platform

Cloud Storage: object storage

Microsoft Azure:

Blob Storage: object storage

ML Studio: ML platform

Synapse: analytics

Cloud ML Services

```
# AWS SageMaker example
import boto3, sagemaker
from sagemaker.sklearn import SKLearn

role = sagemaker.get_execution_role()
sk_estimator = SKLearn(
    entry_point='train.py',
    role=role, instance_type='ml.m5.xlarge',
    framework_version='0.23-1'
)
sk_estimator.fit({'train': 's3://bucket/train.csv'})

# Deploy model
predictor = sk_estimator.deploy(
    initial_instance_count=1, instance_type='ml.t2.medium'
)
```

Data Pipelines

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-team',
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

with DAG('etl_pipeline', default_args=default_args,
        schedule_interval='@daily', start_date=datetime(2024,1,1)) as dag:
    extract = PythonOperator(task_id='extract', python_callable=extract_data)
    transform = PythonOperator(task_id='transform', python_callable=transform_data)
    load = PythonOperator(task_id='load', python_callable=load_data)
    extract >> transform >> load
```